SESSION 10

Software Metrics & Complexity Analysis

CareFrame – Digital Health Record Management System

Deliverable	Metrics Calculation Report
Industry Mapping	Software Quality Measurement
Topics	Size-Oriented Metrics • Halstead Metrics • Function Point Analysis • Cyclomatic Complexity • Control Flow Graphs

Academic worked examples. Numerical values are illustrative assumptions, not measurements extracted from the CareFrame source-code repository.

1. Introduction

Software metrics provide quantitative measures of software size, functionality, complexity, productivity, and quality. This report applies common measurement techniques to CareFrame, a digital health record management system with patient and doctor portals, medical record management, report processing, AI-assisted summaries, and administrative monitoring.

Important: The calculations are illustrative. Actual LOC, token counts, function-point classifications, and CFG measurements have not been extracted from the project repository.

2. Size-Oriented Metrics

Size-oriented metrics estimate the amount of software produced. Common measures include Lines of Code (LOC), Thousands of Lines of Code (KLOC), productivity, and defect density.

2.1 Estimated module-wise LOC

CareFrame module	Estimated LOC
Authentication and RBAC	350
Patient Management	400
Doctor Management	300
Medical Records	550
Report Processing	450
AI Summary Generation	500
Admin and Audit Logs	300
Frontend Components	900
Total	3,750

Total LOC = 3,750
KLOC = LOC / 1,000 = 3,750 / 1,000 = 3.75 KLOC

2.2 Productivity example

Assume 5 developers work for 3 months:
Productivity = LOC / (Developers × Months) = 3,750 / (5 × 3) = 250 LOC per person-month.

2.3 Defect density example

If 15 defects are identified:
Defect Density = Defects / KLOC = 15 / 3.75 = 4 defects/KLOC.

Interpretation: LOC and KLOC provide a size baseline. Productivity and defect density are useful only when counting rules, time periods, and defect-reporting practices are consistent.

3. Halstead Metrics Calculation

Halstead metrics estimate program volume, difficulty, effort, and related measures using operator and operand counts.

3.1 Example function

```
function calculateAge(birthYear, currentYear) {
    const age = currentYear - birthYear;
    if (age >= 18) {
        return age;
    }
    return 0;
}
```

For this worked example, assume the following token counts. These counts are illustrative and depend on the chosen token-counting convention.

Symbol	Meaning	Assumed value
n_1	Distinct operators	6
n_2	Distinct operands	5
N_1	Total operator occurrences	8
N_2	Total operand occurrences	9

3.2 Calculations

Program vocabulary: $n = n_1 + n_2 = 6 + 5 = 11$.
Program length: $N = N_1 + N_2 = 8 + 9 = 17$.
Volume: $V = N \times \log_2(n) = 17 \times \log_2(11) \approx 58.82$.
Difficulty: $D = (n_1 / 2) \times (N_2 / n_2) = (6 / 2) \times (9 / 5) = 5.4$.
Effort: $E = D \times V = 5.4 \times 58.82 \approx 317.63$.
Estimated time: $T = E / 18 \approx 17.65$ seconds.
Estimated delivered bugs: $B = V / 3{,}000 \approx 0.0196$.

Interpretation: Halstead values help compare code sections under the same counting convention. Estimated time and bugs are theoretical indicators, not measured runtime or actual defect counts.

4. Function Point Analysis (FPA)

Function Point Analysis estimates functionality delivered to users, independently of programming language. It classifies functions as External Inputs (EI), External Outputs (EO), External Inquiries (EQ), Internal Logical Files (ILF), and External Interface Files (EIF).

4.1 CareFrame function point estimate

Component	CareFrame examples	Count	Low weight	Subtotal
EI	Login, report upload, profile update	8	3	24
EO	AI summary, report export	5	4	20
EQ	Search records, view history	6	3	18
ILF	Users, patients, records, reports, audit logs	5	7	35
EIF	External reference data/interfaces	2	5	10
Total				107

Weights are standard low-complexity weights. Counts and complexity assumptions are illustrative and require validation against detailed requirements.

4.2 Unadjusted Function Points

UFP = 24 + 20 + 18 + 35 + 10 = 107.

4.3 Value Adjustment Factor

VAF = 0.65 + (0.01 × TDI). Assume Total Degree of Influence (TDI) across the 14 general system characteristics is 35.
VAF = 0.65 + (0.01 × 35) = 1.00.

4.4 Adjusted Function Points

FP = UFP × VAF = 107 × 1.00 = 107 Function Points.

Interpretation: The estimate provides a functional-size baseline. A formal FPA assessment should classify each transaction and logical data group by complexity using the selected counting standard.

5. Cyclomatic Complexity

Cyclomatic Complexity measures the number of linearly independent paths through a function. It supports test planning by identifying independent paths needed for basis-path coverage.

5.1 Formula

M = E − N + 2P, where E is the number of edges, N is the number of nodes, and P is the number of connected components. For one connected function, M is also the number of decision points + 1.

5.2 CareFrame example: report access validation

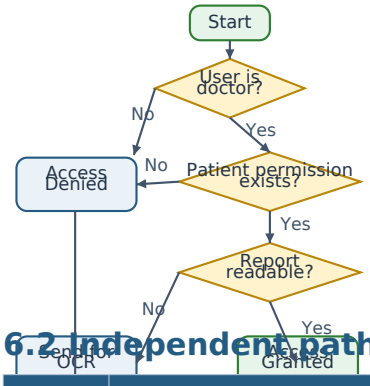
```
function validateReportAccess(user, report) {
    if (user.role !== "doctor") {
        return "Access Denied";
    }
    if (!user.hasPatientPermission) {
        return "Access Denied";
    }
    if (!report.isReadable) {
        return "Send for OCR";
    }
    return "Access Granted";
}
```

There are three decision points: (1) user is a doctor, (2) patient permission exists, and (3) report is readable. M = 3 + 1 = 4. Therefore, four independent paths should be considered in basis-path testing.

6. Control Flow Graph (CFG)

A Control Flow Graph represents execution flow. Nodes represent statements or blocks, directed edges represent possible transfers of control, and decision nodes represent conditions.

6.1 CFG for validateReportAccess()



6.2 Independent paths

Path	Execution	Outcome
P1	Start → Doctor? No → Denied → End	Access denied
P2	Start → Doctor? Yes → Permission? No → Denied → End	Access denied
P3	Start → Doctor? Yes → Permission? Yes → Readable? No → OCR → End	OCR processing
P4	Start → Doctor? Yes → Permission? Yes → Readable? Yes → Granted → End	Access granted

6.3 Verification using CFG formula

For this graph: E = 10, N = 8, P = 1.
M = E − N + 2P = 10 − 8 + 2(1) = 4.
The result matches the decision-point calculation. Node and edge counts depend on how statements are grouped into basic blocks.

7. Overall Metrics Summary

Metric	Illustrative result	Purpose
LOC	3,750	Measures software size
KLOC	3.75	Standardized size measurement
Productivity	250 LOC/person-month	Development productivity
Defect Density	4 defects/KLOC	Tracks defects relative to size
Halstead Volume	58.82	Program volume
Halstead Difficulty	5.4	Estimated programming difficulty
Halstead Effort	317.63	Theoretical implementation effort
Function Points	107 FP	Functional size estimate
Cyclomatic Complexity	4	Independent execution paths

8. Industry Mapping – Software Quality Measurement

Industry practice	Metric / technique	CareFrame application
Software Size Estimation	LOC, KLOC	Estimate project size and track growth
Productivity Measurement	LOC/person-month	Compare development output over time
Defect Measurement	Defect density	Track reported defects per KLOC
Code Complexity Analysis	Halstead metrics	Compare complexity across functions
Functional Size Estimation	Function Points	Estimate user-visible functionality
Test Planning	Cyclomatic complexity	Identify independent paths for testing
Control Flow Testing	CFG	Design branch and basis-path test cases

9. Conclusion

Software metrics provide a quantitative approach to evaluating CareFrame’s size, functionality, complexity, and quality. Size-oriented metrics estimate implementation scale; Halstead metrics characterize code using operator and operand counts; Function Point Analysis estimates user-visible functionality; and Cyclomatic Complexity identifies independent execution paths. Control Flow Graphs support systematic test-case design and help developers reason about branches and outcomes.

Together, these techniques support software quality measurement, maintainability, test planning, and project estimation. The numerical examples are illustrative and should be replaced with measured repository data before being presented as empirical project results.

10. Measurement Notes and Assumptions

• LOC totals are estimated module allocations, not measured source-code counts.
• Productivity assumes 5 developers and 3 months of work.
• Defect density assumes 15 recorded defects.
• Halstead token counts are assumed for demonstration; parsers may classify tokens differently.
• Function Point counts and complexity weights are illustrative. Formal FPA requires detailed transaction and data-function classification.
• The CFG example is a single connected function with three decisions and four independent paths.